

程式碼實驗室：瞭解如何在 Cloud Run 建構及部署 LangChain 應用程式

來源：<https://codelabs.developers.google.com/codelabs/build-and-deploy-a-langchain-app-on-cloud-run?hl=zh-tw>

步驟數：9

瞭解如何在 Cloud Run 中建構及部署 LangChain 應用程式

目錄

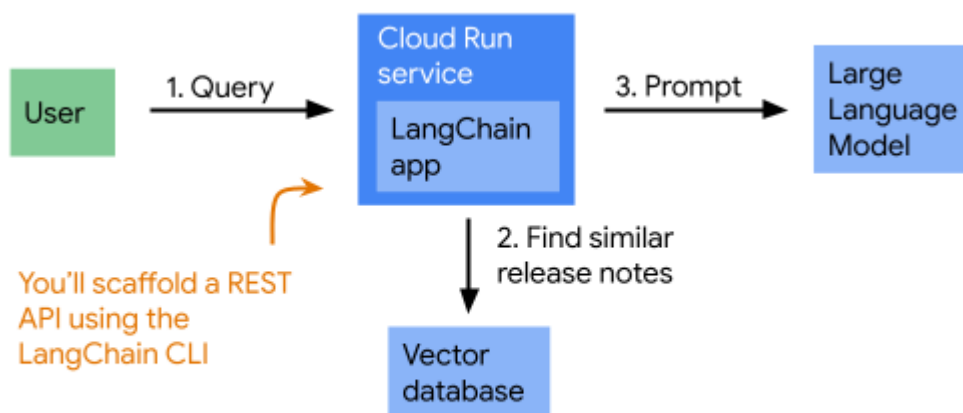
1. [總覽](#)
2. [設定和需求](#)
3. [建立向量資料庫執行個體](#)
4. [架構 LangChain 應用程式](#)
5. [建立索引工作](#)
6. [編寫網頁應用程式](#)
7. [將網頁應用程式部署至 Cloud Run](#)
8. [探索遊樂場](#)
9. [恭喜](#)

1. 總覽

在本程式碼研究室中，您將瞭解如何部署使用 Gemini 的 LangChain 應用程式，以便查詢 Cloud Run 版本資訊。

以下是應用程式的運作方式範例：如果你問「我可以在 Cloud Run 中將 Cloud Storage 值區掛接為磁碟區嗎？」，應用程式會回覆「可以，自 2024 年 1 月 19 日起即可」或類似內容。

為傳回有根據的回覆，應用程式會先擷取與問題類似的 Cloud Run 版本說明，然後同時將問題和版本說明提供給 Gemini。(這就是一般所稱的 RAG 模式)。下圖顯示應用程式的架構：



步驟 2 · 約 0 分鐘

2. 設定和需求

首先，請確認開發環境設定正確無誤。

- 您需要 **Google Cloud 專案**，才能部署應用程式所需的資源。
- 如要部署應用程式，您必須在本機安裝 **gcloud**、**完成驗證**，並設定使用專案。
 - `gcloud auth login`
 - `gcloud config set project`
- 如要在本機執行應用程式 (建議您這麼做)，請務必正確設定**應用程式預設憑證**，包括設定配額專案。
 - `gcloud auth application-default login`
 - `gcloud auth application-default set-quota-project`
- 您也必須安裝下列軟體：
 - **Python** (必須為 3.11 以上版本)
 - [LangChain CLI](#)
 - 使用 [poetry](#) 管理依附元件
 - [pipx](#)，可在獨立虛擬環境中安裝及執行 LangChain CLI 和 poetry

請參閱這篇[網誌](#)，瞭解如何開始安裝本逐步解說所需的工具。

Cloud Workstations

您也可以使用 Google Cloud 上的 Cloud Workstations，不必在本機上執行。請注意，截至 2024 年 4 月，該服務執行的 Python 版本低於 3.11，因此您可能需要先升級 Python，才能開始使用。

啟用 Cloud API

首先，請執行下列指令，確認您已設定要使用的正確 Google Cloud 專案：

〇〇〇

```
gcloud config list project
```

如未顯示正確的專案，請輸入下列指令手動設定專案：

〇〇〇

```
gcloud config set project <PROJECT_ID>
```

現在請啟用下列 API：

〇〇〇

```
gcloud services enable \  
  bigquery.googleapis.com \  
  sqladmin.googleapis.com \  
  aiplatform.googleapis.com \  
  cloudresourcemanager.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  run.googleapis.com \  
  secretmanager.googleapis.com
```

選擇區域

Google Cloud 服務遍及全球各地，您必須選擇一個位置，部署本實驗室使用的資源。在殼層中將區域設為環境變數 (後續指令會使用這個變數)：

```
export REGION=us-central1
```

3. 建立向量資料庫執行個體

這項應用程式的重點是擷取與使用者問題相關的版本說明。舉例來說，如果您要詢問 Cloud Storage 相關問題，希望系統在提示中加入下列版本資訊：

2024 年 1 月 19 日：您現在可以將 Cloud Storage bucket 掛接為服務的儲存空間磁碟區，以及 Cloud Run 工作。

您可以使用文字嵌入和向量資料庫，找出語意相似的版本說明。

我將說明如何使用 Cloud SQL 上的 PostgreSQL 做為向量資料庫。建立新的 Cloud SQL 執行個體需要一些時間，因此請立即建立。

```
gcloud sql instances create sql-instance \  
  --database-version POSTGRES_14 \  
  --tier db-f1-micro \  
  --region $REGION
```

您可以讓這項指令繼續執行，然後進行後續步驟。您稍後需要建立資料庫並新增使用者，但現在先別浪費時間等待微調器。

PostgreSQL 是關聯式資料庫伺服器，而每個新的 Cloud SQL 執行個體預設都會安裝 [pgvector](#) 擴充功能，因此您也可以將其做為向量資料庫。

步驟 4 · 約 0 分鐘

4. 架構 LangChain 應用程式

如要繼續操作，請先安裝 LangChain CLI，並使用 Poetry 管理依附元件。使用 pipx 安裝這些工具的方法如下：

```
pipx install langchain-cli poetry
```

使用下列指令架構 LangChain 應用程式。系統詢問時，請將資料夾命名為 `run-rag`，然後按下 Enter 鍵，略過安裝套件：

```
langchain app new
```

切換至 `run-rag` 目錄並安裝依附元件

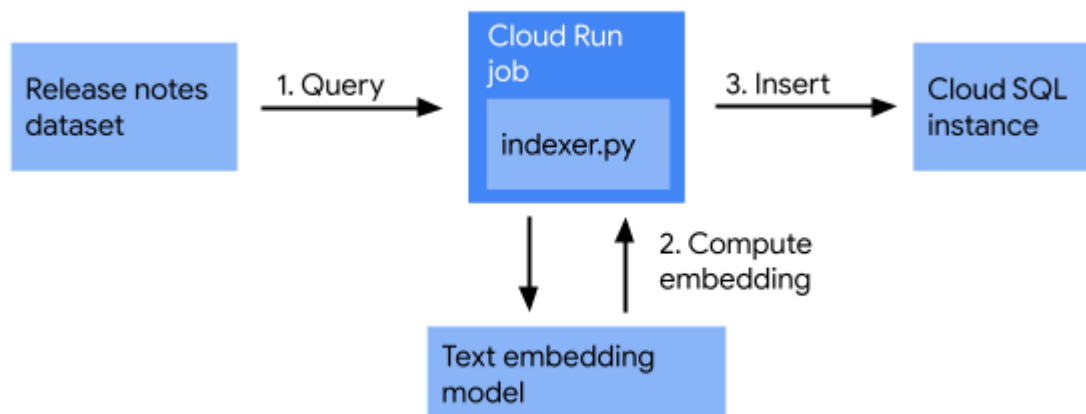
```
poetry install
```

您剛建立 [LangServe](#) 應用程式。LangServe 會將 FastAPI 包裝在 LangChain 鏈結周圍。內建的遊樂場可讓您輕鬆傳送提示及檢查結果，包括所有中間步驟。建議您在編輯器中開啟 `run-rag` 資料夾，並探索其中的內容。

步驟 5 · 約 0 分鐘

5. 建立索引工作

開始建構網頁應用程式之前，請先確認 Cloud SQL 資料庫已為 Cloud Run 版本資訊建立索引。在本節中，您將建立索引工作，執行下列操作：



建立索引工作會擷取版本說明，使用文字嵌入模型將其轉換為向量，然後儲存在向量資料庫中。這樣就能根據語意快速搜尋類似的發行說明。

在 `run-rag/app` 資料夾中，建立包含下列內容的 `indexer.py` 檔案：


```

import os
from google.cloud.sql.connector import Connector
import pg8000
from langchain_community.vectorstores.pgvector import PGVector
from langchain_google_vertexai import VertexAIEmbeddings
from google.cloud import bigquery

# Retrieve all Cloud Run release notes from BigQuery
client = bigquery.Client()
query = """
SELECT
    CONCAT(FORMAT_DATE("%B %d, %Y", published_at), ": ", description) AS release_note
FROM `bigquery-public-data.google_cloud_release_notes.release_notes`
WHERE product_name= "Cloud Run"
ORDER BY published_at DESC
"""
rows = client.query(query)

print(f"Number of release notes retrieved: {rows.result().total_rows}")

# Set up a PGVector instance
connector = Connector()

def getconn() -> pg8000.dbapi.Connection:
    conn: pg8000.dbapi.Connection = connector.connect(
        os.getenv("DB_INSTANCE_NAME", ""),
        "pg8000",
        user=os.getenv("DB_USER", ""),
        password=os.getenv("DB_PASS", ""),
        db=os.getenv("DB_NAME", ""),
    )
    return conn

store = PGVector(
    connection_string="postgresql+pg8000://",
    use_jsonb=True,
    engine_args=dict(
        creator=getconn,
    ),
    embedding_function=VertexAIEmbeddings(
        model_name="textembedding-gecko@003"
    ),
    pre_delete_collection=True
)

# Save all release notes into the Cloud SQL database
texts = list(row["release_note"] for row in rows)
ids = store.add_texts(texts)

print(f"Done saving: {len(ids)} release notes")

```

新增必要的依附元件：

```
poetry add \  
  "cloud-sql-python-connector[pg8000]" \  
  langchain-google-vertexai==1.0.5 \  
  langchain-community==0.2.5 \  
  pgvector
```

建立資料庫和使用者

在 Cloud SQL 執行個體 `sql-instance` 上建立資料庫 `release-notes`：

```
gcloud sql databases create release-notes --instance sql-instance
```

建立名為 `app` 的資料庫使用者：

```
gcloud sql users create app --instance sql-instance --password "myprecious"
```

注意：為了方便說明，我在此使用容易猜到的密碼。在實驗室以外的環境中，強烈建議您產生隨機密碼、將密碼儲存在 Secret Manager 中，並將密鑰公開為應用程式的環境變數 (或掛接為檔案)。

部署及執行索引工作

現在請部署及執行工作：

```
DB_INSTANCE_NAME=$(gcloud sql instances describe sql-instance --  
format="value(connectionName)")  
  
gcloud run jobs deploy indexer \  
  --source . \  
  --command python \  
  --args app/indexer.py \  
  --set-env-vars=DB_INSTANCE_NAME=$DB_INSTANCE_NAME \  
  --set-env-vars=DB_USER=app \  
  --set-env-vars=DB_NAME=release-notes \  
  --set-env-vars=DB_PASS=myprecious \  
  --region=$REGION \  
  --execute-now
```

這個指令很長，讓我們看看發生了什麼事：

第一個指令會擷取連線名稱 (格式為 `project:region:instance` 的不重複 ID)，並將其設為 `DB_INSTANCE_NAME` 環境變數。

第二個指令會部署 Cloud Run 工作。這些標記的作用如下：

- `--source .`：指定作業的原始碼位於目前工作目錄 (您執行指令的目錄)。

- `--command python`：設定要在容器內執行的指令。在本例中，這是指執行 Python。
- `--args app/indexer.py`：提供 python 指令的引數。這會指示在應用程式目錄中執行 script indexer.py。
- `--set-env-vars`：設定 Python 指令碼在執行期間可存取的環境變數。
- `--region=$REGION`：指定要部署工作的區域。
- `--execute-now`：告知 Cloud Run 在部署工作後立即啟動。

如要確認工作是否順利完成，請執行下列操作：

- 透過網頁控制台讀取工作執行的記錄。系統應會回報「已儲存：xxx 則版本資訊」（其中 xxx 是儲存的版本資訊數量）。
 - 您也可以透過網頁版控制台中前往 Cloud SQL 執行個體，然後使用 [Cloud SQL Studio](#) 查詢 `langchain_pg_embedding` 資料表中的記錄數。
-

步驟 6 · 約 0 分鐘

6. 編寫網頁應用程式

在編輯器中開啟 `app/server.py` 檔案。你會看到如下所示的行：

```
# Edit this to add the chain you want to add
```

將該註解替換為下列程式碼片段：

```

# (1) Initialize VectorStore
connector = Connector()

def getconn() -> pg8000.dbapi.Connection:
    conn: pg8000.dbapi.Connection = connector.connect(
        os.getenv("DB_INSTANCE_NAME", ""),
        "pg8000",
        user=os.getenv("DB_USER", ""),
        password=os.getenv("DB_PASS", ""),
        db=os.getenv("DB_NAME", "")
    )
    return conn

vectorstore = PGVector(
    connection_string="postgresql+pg8000://",
    use_jsonb=True,
    engine_args=dict(
        creator=getconn,
    ),
    embedding_function=VertexAIEmbeddings(
        model_name="textembedding-gecko@003"
    )
)

# (2) Build retriever

def concatenate_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

notes_retriever = vectorstore.as_retriever() | concatenate_docs

# (3) Create prompt template
prompt_template = PromptTemplate.from_template(
    """You are a Cloud Run expert answering questions.
Use the retrieved release notes to answer questions
Give a concise answer, and if you are unsure of the answer, just say so.

Release notes: {notes}

Here is your question: {query}
Your answer: """
)

# (4) Initialize LLM
llm = VertexAI(
    model_name="gemini-1.0-pro-001",
    temperature=0.2,
    max_output_tokens=100,
    top_k=40,

```

```

        top_p=0.95
    )

# (5) Chain everything together
chain = (
    RunnableParallel({
        "notes": notes_retriever,
        "query": RunnablePassthrough()
    })
    | prompt_template
    | llm
    | StrOutputParser()
)

```

您也需要新增下列匯入項目：

```

import pg8000
import os
from google.cloud.sql.connector import Connector
from langchain_google_vertexai import VertexAI
from langchain_google_vertexai import VertexAIEmbeddings
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_community.vectorstores.pgvector import PGVector

```

最後，將顯示「NotImplemented」的行變更為：

```

# add_routes(app, NotImplemented)
add_routes(app, chain)

```

步驟 7 · 約 0 分鐘

7. 將網頁應用程式部署至 Cloud Run

從 `run-rag` 目錄使用下列指令，將應用程式部署至 Cloud Run：

```
DB_INSTANCE_NAME=$(gcloud sql instances describe sql-instance --
format="value(connectionName)")

gcloud run deploy run-rag \
  --source . \
  --set-env-vars=DB_INSTANCE_NAME=$DB_INSTANCE_NAME \
  --set-env-vars=DB_USER=app \
  --set-env-vars=DB_NAME=release-notes \
  --set-env-vars=DB_PASS=myprecious \
  --region=$REGION \
  --allow-unauthenticated
```

這項指令會執行以下作業：

- 將原始碼上傳至 Cloud Build
- 執行 docker build。
- 將產生的容器映像檔推送至 Artifact Registry。
- 使用容器映像檔建立 Cloud Run 服務。

指令執行完成後，系統會列出 run.app 網域上的 HTTPS 網址。這是新 Cloud Run 服務的公開網址

8. 探索遊樂場

開啟 Cloud Run 服務網址，然後前往 `/playground`。系統會顯示文字欄位。你可以用它來詢問 Cloud Run 版本資訊的相關問題，例如：

 **LangServe Playground**

Try it

Inputs

Reset

RUNNABLEPARALLEL<NOTES,QUERY>INPUT
When did Cloud Run launch Cloud Storage mounts?

Output

January 19, 2024

Intermediate steps 8 >

Share

Start

9. 恭喜

您已成功在 Cloud Run 建構及部署 LangChain 應用程式。非常好！

以下是幾個重要概念：

- 使用 LangChain 架構建構檢索增強生成 (RAG) 應用程式。
- 使用 Cloud SQL 上的 PostgreSQL 做為向量資料庫，並搭配 pgvector (Cloud SQL 預設會安裝此擴充功能)。
- 以 Cloud Run 工作形式執行較長時間的索引工作，並以 Cloud Run 服務形式執行網頁應用程式。
- 使用 LangServe 將 LangChain 鏈結包裝在 FastAPI 應用程式中，提供方便的介面與 RAG 應用程式互動。

清除所用資源

如要避免系統向您的 Google Cloud Platform 帳戶收取您在本教學課程中所用資源的相關費用：

- 前往 Cloud Console 中的「管理資源」頁面。
- 在專案清單中選取專案，然後按一下「刪除」。
- 在對話方塊中輸入專案 ID，然後按一下「關閉」即可刪除專案。

如要保留專案，請務必刪除下列資源：

- Cloud SQL 執行個體
 - Cloud Run 服務
 - Cloud Run 工作
-